# FIT3155: Solutions to conceptual questions in Week 2 Lab sheet

1. Consider the following string:

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| a | b | b | a | b | c | a | b | b | a | b | b | a | b | a | b | c |

Work out (on paper) the mechanics of the $Z$-algorithm when run on this string. While doing so, amongst others, identify which cases (Case 1, Case 2a, Case 2b) apply when computing $Z[k]$ at positions $k = \{7, 10, 13, 15\}$.

**Ans. 1**

| $k$ | $[l, r]_{k-1}$ | **Case** | $Z[k]$ |
|---|---|---|---|
| 2 | [0, 0] | Case 1 $(\underbrace{k}_{2} > \underbrace{r}_{0})$ | 0 |
| 3 | [0, 0] | Case 1 $(\underbrace{k}_{3} > \underbrace{r}_{0})$ | 0 |
| 4 | [0, 0] | Case 1 $(\underbrace{k}_{4} > \underbrace{r}_{0})$ | 2 |
| 5 | [4, 5] | Case 2a $(\underbrace{k}_{5} \leq \underbrace{r}_{5}$ and $\underbrace{Z[k-l+1]}_{Z[2]=0} < \underbrace{r-k+1}_{1})$ | 0 |
| 6 | [4, 5] | Case 1 $(\underbrace{k}_{6} > \underbrace{r}_{5})$ | 0 |
| 7 | [4, 5] | Case 1 $(\underbrace{k}_{7} > \underbrace{r}_{5})$ | 5 |
| 8 | [7, 11] | Case 2a $(\underbrace{k}_{8} \leq \underbrace{r}_{11}$ and $\underbrace{Z[k-l+1]}_{Z[2]=0} < \underbrace{r-k+1}_{4})$ | 0 |
| 9 | [7, 11] | Case 2a $(\underbrace{k}_{9} \leq \underbrace{r}_{11}$ and $\underbrace{Z[k-l+1]}_{Z[3]=0} < \underbrace{r-k+1}_{3})$ | 0 |
| 10 | [7, 11] | Case 2b $(\underbrace{k}_{10} \leq \underbrace{r}_{11}$ and $\underbrace{Z[k-l+1]}_{Z[4]=2} \geq \underbrace{r-k+1}_{2})$ | 5 |
| 11 | [10, 14] | Case 2a $(\underbrace{k}_{11} \leq \underbrace{r}_{14}$ and $\underbrace{Z[k-l+1]}_{Z[2]=0} < \underbrace{r-k+1}_{4})$ | 0 |
| 12 | [10, 14] | Case 2a $(\underbrace{k}_{12} \leq \underbrace{r}_{14}$ and $\underbrace{Z[k-l+1]}_{Z[3]=0} < \underbrace{r-k+1}_{3})$ | 0 |
| 13 | [10, 14] | Case 2b $(\underbrace{k}_{13} \leq \underbrace{r}_{14}$ and $\underbrace{Z[k-l+1]}_{Z[4]=2} \geq \underbrace{r-k+1}_{2})$ | 2 |
| 14 | [13, 14] | Case 2a $(\underbrace{k}_{14} \leq \underbrace{r}_{14}$ and $\underbrace{Z[k-l+1]}_{Z[2]=0} < \underbrace{r-k+1}_{1})$ | 0 |
| 15 | [13, 14] | Case 1 $(\underbrace{k}_{15} > \underbrace{r}_{14})$ | 2 |
| 16 | [15, 16] | Case 2a $(\underbrace{k}_{16} \leq \underbrace{r}_{16}$ and $\underbrace{Z[k-l+1]}_{Z[2]=0} < \underbrace{r-k+1}_{1})$ | 0 |
| 17 | [15, 16] | Case 1 $(\underbrace{k}_{17} > \underbrace{r}_{16})$ | 0 |

2. For a string $S[1 \ldots N]$ (i.e., reasoning in 1-based indexing), suppose we observed that $Z[2] = Len > 0$. Reason why the values of $Z[k]$ for $3 \leq k \leq Len + 2$ can be resolved without additional explicit character comparisons.

3. Now reason which specific case(s) of the Z-algorithm (Case 1, Case 2a, or Case 2b) apply when computing those $Z[k]$ values.

**Ans. 2**
$Z[2] = Len > 0$ implies (by the definition of the Z-box at position $k = 2$):

$$\begin{aligned} S[2] &= S[1] \\ S[3] &= S[2] \\ S[4] &= S[3] \\ &\vdots \\ S[Len+1] &= S[Len] \\ S[Len+2] &\neq S[Len+1]. \end{aligned}$$

The above pairwise relationships between successive characters will constrain $S[1], S[2], \ldots, S[Len+1]$ to be all identical to each other. This run of single-character repeats of length $Len + 1$ ends at position $Len+2$ where there is a different character. Since each $Z[k]$ value gives the longest substring starting at a position $k$ that matches the prefix of the string of the same length, the constraints automatically yield $Z[k] = Len - k + 2 \quad \forall\, 3 \leq k \leq Len + 2$.

**Ans. 3**
$[l, r]$ values after computing $Z[2]$ will be $[2, Len + 1]$. For all $3 \leq k \leq Len + 1$, the case table is shown below:

| $k$ | $[l, r]_{k-1}$ | **Case** | $Z[k]$ |
|---|---|---|---|
| 3 | $[2,\ Len+1]$ | Case 2b $(\underbrace{k}_{3} \leq \underbrace{r}_{Len+1}$ and $\underbrace{Z[k-l+1]}_{Z[2]=Len} \geq \underbrace{r-k+1}_{Len-1})$ | $Len - 1$ |
| 4 | $[2,\ Len+1]$ | Case 2b $(\underbrace{k}_{4} \leq \underbrace{r}_{Len+1}$ and $\underbrace{Z[k-l+1]}_{Z[3]=Len-1} \geq \underbrace{r-k+1}_{Len-2})$ | $Len - 2$ |
| $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ |
| $k$ | $[2,\ Len+1]$ | Case 2b (Pattern continues: $\underbrace{Z[k-l+1]}_{=Len-(k-3)} \geq \underbrace{r-k+1}_{=Len-(k-3)-1}$) | $Len - k + 2$ |
| $\vdots$ | $\vdots$ | $\vdots$ | $\vdots$ |
| $Len$ | $[2,\ Len+1]$ | Case 2b $(\underbrace{k}_{Len} \leq \underbrace{r}_{Len+1}$ and $\underbrace{Z[k-l+1]}_{Z[Len-1]=2} \geq \underbrace{r-k+1}_{1})$ | 2 |
| $Len + 1$ | $[2,\ Len+1]$ | Case 2b $(\underbrace{k}_{Len+1} \leq \underbrace{r}_{Len+1}$ and $\underbrace{Z[k-l+1]}_{Z[Len]=1} \geq \underbrace{r-k+1}_{1})$ | 1 |
| $Len + 2$ | $[2,\ Len+1]$ | Case 1 $(\underbrace{k}_{Len+2} > \underbrace{r}_{Len+1})$ | 0 |

4. Case 2b of the $Z$-algorithm (i.e., the case handling the scenario where $k \leq r$ and $Z[k-l+1] \geq r-k+1$) can be further decomposed into two distinct cases:

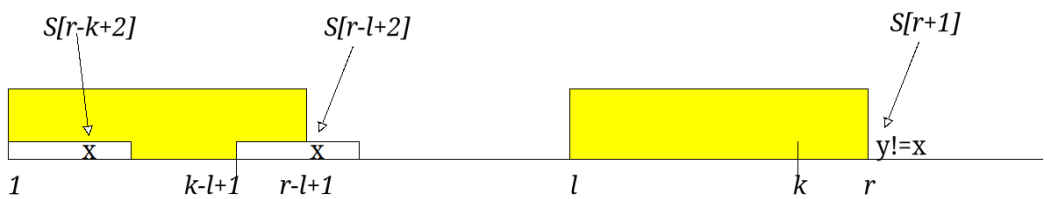
   **case 2b.1** $Z[k-l+1] > r-k+1$

   **case 2b.2** $Z[k-l+1] = r-k+1$

   Justify that when $Z[k-l+1] > r-k+1$, then one can immediately conclude that $Z[k] = r-k+1$ without needing any explicit character comparisons. In other words, explicit character comparisons are only needed for $Z[k-l+1] = r-k+1$ case.

   **Ans. 4**

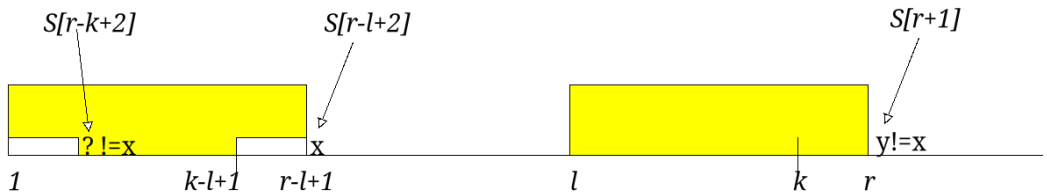
   **Case 2b.1**




   - From the definition of $Z[l]$:
     - $S[l \ldots r] = S[1 \ldots r-l+1]$, and
     - $S[r+1] \neq S[r-l+2]$.
   - From the definition of $Z[k-l+1] = Len$ (say), we know that $S[k-l+1 \ldots k-l+Len] = S[1 \ldots Len]$.
   - $Len > r-k+1 \implies S[r-l+2] = S[r-k+2]$.
   - Thus, we have
     - $S[k \ldots r] = S[k-l+1 \ldots r-l+1]$, (given the definition of $Z[l]$)
     - $S[k-l+1 \ldots r-l+1] = S[1 \ldots r-k+1]$, (implicit from the definition of $Z[k-l+1]$), and
     - $S[r-l+2] = S[r-k+2] \neq S[r+1]$

     This is sufficient to conclude that $Z[k] = r-k+1$ (without any explicit comparisons).

**Case 2b.2**




- By the definition of $Z[l]$ box, we have:
  - $S[l \ldots r] = S[1 \ldots r - l + 1]$, and
  - $S[r - l + 2] \neq S[r + 1]$.
- If $Z[k-l+1] = r-k+1$, this implies $S[r-l+2] \neq S[r-k+2]$.
- At this stage all we can conclude is that:
  - $S[k \ldots r] = S[k - l + 1 \ldots r - l + 1]$, and
  - $S[k - l + 1 \ldots r - l + 1] = S[1 \ldots r - k + 1]$.
  - $S[r - k + 2] \neq S[r - l + 2]$

  However, we cannot conclude whether the characters $S[r + 1]$ and $S[r - k + 2]$ are the same (match) or not (mismatch). Therefore, to determine $Z[k]$ precisely, explicit comparisons are required starting from the comparison of $S[r + 1]$ with $S[r - k + 2]$ until the first mismatch is found.

5. Prove that the $Z$-algorithm runs in $O(N)$ time requiring $O(N)$ space in the worst case, where $N$ is the length of the input string.

   **Ans. 5**
   Refer lecture slides.

6. Given two strings of the same length $S[1 \ldots N]$ and $T[1 \ldots N]$ we want to determine if $S$ is a cyclic rotation of $T$ (or vice versa). How would be approach solving this problem and what is its worst-case time complexity?

   **Ans. 6**
   Doubling $S$ will allow us to check if $T$ is a substring of doubled $S$ solvable as a linear-time pattern matching problem using $Z$-algorithm as follows:

```
1 U[1...3N+1] = T + '$' + S + S
2 Z = zalg(U)
3 looping k from N+2 to 2N+1
4   if (Z[k] == N)
5     return S is a cyclic rotation of T
6 return S is NOT a cyclic rotation of T
```

   In the above pseudocode:

   - Line 1 constructing $U$ via string concatenations requires $O(N)$ effort.
   - Line 2 constructing $Z$ values of $U$ also takes $O(N)$ effort (as per proof of Q5).
   - Loop starting line 3 iterates at most N times, with each iteration requiring $O(1)$ effort.

   This yields the worst-case time bounded by $O(N)$.

7. Write pseudocode for an algorithm that takes two strings $S[1 \ldots M]$ and $T[1 \ldots N]$ and returns the length of the longest suffix of $S$ that exactly matches a prefix of $T$.* What is the worst-case time complexity of your algorithm?

**Ans. 7**

```
1 U[1...N+M+1] = T + '$' + S
2 Z = zalg(U)
3 looping k from N+2 to N+M+1
4   if (k+Z[k]-1 == N+M+1)
5     return Z[k]
6 return 0
```

In the above pseudocode:

- Line 1 constructing $U$ requires $O(N + M)$ effort.
- Line 2 constructing $Z$ values of $U$ also takes $O(N + M)$ effort.
- Loop starting line 3 iterated at most M times, with each iteration requiring $O(1)$ effort.

Thus, the worst case time is bounded by $O(N + M)$.

*Assume `zalg` is a built-in function available for you, that takes a string as an argument and returns an array of its $Z$-values.

8. Given a string $S[1 \ldots N]$, write pseudocode for an algorithm that computes an array $Z^{\texttt{suffix}}[1 \ldots N]$, where each $Z^{\texttt{suffix}}[i]$ stores the length of the **longest** substring of $S$ **ending** at position $i$ that matches a **suffix** of $S$ of the same length.* Reason the worst-case time complexity of your algorithm.

   **Ans. 8**

```
1 revS = reverse(S)
2 revZsuffix = zalg(revS)
3 return reverse(revZsuffix);
```

   In the above pseudocode:

   - Line 1 requires $O(N)$ effort – this is string reveral.
   - Line 2 constructing $Z$ values of a string, so this takes $O(N)$ effort.
   - Line 3 is reversing the elements of the array returned by line 2, requiring $O(N)$ effort.

   Thus, the worst case time is bounded by $O(N)$.

9. Given a string $S[1 \ldots N]$, write pseudocode for an algorithm that computes an array $\texttt{MP}[1 \ldots N]$, where each $\texttt{MP}[i]$ stores the length of the largest suffix of the string $S[i \ldots N]$ that matches the prefix of $S$.* Reason the worst-case time complexity of your algorithm.

   **Ans. 9**

```
1 Z = zalg(S)
2 MP[1...N+1] = {0...0} //assume MP[N+1] is some
     dummy cell in the array
3 looping i from N backwards down to 2
4   if (i+Z[i]-1 == N)
5     MP[i] = Z[i];
6   else MP[i] = MP[i+1]
7 return MP
```

   In the above pseudocode:

   - Line 1 constructing $Z$ values of a string, so this takes $O(N)$ effort (as per proof of Q5).
   - Line 2 initializes MP taking $O(N)$ effort.
   - Loop starting line 3 iterates $N-1$ times and each iteration requires $O(1)$ effort.

   Thus, the worst case time is bounded by $O(N)$.

```
-=0=-
 END
-=0=-
```